

Deep Dive into Python OOP: How Inheritance Affects the Design of Model Compression

202311294 Park Seonggwon

1. Introduction

In recent years, model compression has become important for running neural networks efficiently on resource-constrained devices. However, implementing multiple compression methods in one software system can easily produce highly coupled and difficult-to-maintain code. This report argues that inheritance, a core concept of Python OOP, provides a scalable and modular way to organize such behaviors.

2. Inheritance

Inheritance is one of the core mechanisms of object-oriented programming in Python. It allows a child class to reuse the attributes and methods of a parent class while also specializing behavior when necessary. This specialization is achieved through method overriding, in which a child class redefines an inherited method to provide its own implementation. As a result, multiple objects can share a common interface while producing different behaviors depending on their class type.

In Python, inheritance is especially useful when several components have the same basic structure but require different internal logic. A parent class can define common data and a required interface, while child classes implement specific behaviors in their own way. The `super()` function is also important because it allows a child class to initialize or extend the parent class without duplicating code. Therefore, inheritance is not only a tool for code reuse, but also a design strategy that improves extensibility, readability, and maintainability. In large systems, new functionality can be added by creating a new subclass rather than modifying stable existing code, which supports the Open-Closed Principle and makes the software safer to evolve.

3. Implementation

The experiments were conducted by implementing a `forward` function representing a simplified neural network layer. Experiment 1 uses a procedural approach, whereas Experiment 2 implements the same functionality using inheritance and polymorphism.

To make the comparison fair, both experiments use the same input value $\mathbf{x} = 2.0$ and the same weight list $[0.1, -0.8, 0.4, -0.2]$. In the pruning case, the threshold is set to 0.3 , which means that weights whose absolute values are smaller than or equal to 0.3 are replaced with 0.0 . In the quantization case, each weight is rounded to the nearest integer to simulate reduced numerical precision. Because both experiments use the same settings, any difference comes from the programming approach itself.

A. Experiment 1: Procedural Approach

```
def forward(x, weights, compression, threshold=0.0):  
    if compression == "basic":  
        return [w * x for w in weights]  
  
    elif compression == "pruning":  
        return [(w if abs(w) > threshold else 0.0) * x for w in weights]  
  
    elif compression == "quantization":  
        return [round(w) * x for w in weights]  
  
    else:  
        raise ValueError("Unknown layer type")  
  
weights = [0.1, -0.8, 0.4, -0.2]  
  
print(f"basic forward : {forward(2.0, weights, \"basic\", threshold=0.3)}")  
print(f"pruning forward : {forward(2.0, weights, \"pruning\", threshold=0.3)}")  
print(f"quantization forward : {forward(2.0, weights, \"quantization\", threshold=0.3)}")
```

(a) Code for the procedural approach.

```
basic forward : [0.2, -1.6, 0.8, -0.4]  
pruning forward : [0.0, -1.6, 0.8, 0.0]  
quantization forward : [0.0, -2.0, 0.0, 0.0]
```

(b) Execution result of the procedural approach.

Figure 1: Procedural implementation of the toy compression layer and its output.

B. Experiment 2: Object-Oriented Approach (Inheritance)

```
class BaseLayer:
    def __init__(self, name, weights):
        self.name = name
        self.weights = weights

    def forward(self, x):
        raise NotImplementedError("Subclasses must implement forward method.")

class BasicLayer(BaseLayer):
    def forward(self, x):
        return [w * x for w in self.weights]

class PrunedLayer(BaseLayer):
    def __init__(self, name, weights, threshold=0.5):
        super().__init__(name, weights)
        self.threshold = threshold

    def forward(self, x):
        return [(w if abs(w) > self.threshold else 0.0) * x for w in self.weights]

class QuantizedLayer(BaseLayer):
    def forward(self, x):
        return [round(w) * x for w in self.weights]

layers = [
    BasicLayer("basic forward", [0.1, -0.8, 0.4, -0.2]),
    PrunedLayer("pruning forward", [0.1, -0.8, 0.4, -0.2], threshold=0.3),
    QuantizedLayer("quantization forward", [0.1, -0.8, 0.4, -0.2])
]

for layer in layers:
    print(f"{layer.name} : {layer.forward(2.0)}")
```

(a) Code for the inheritance-based approach.

```
basic forward : [0.2, -1.6, 0.8, -0.4]
pruning forward : [0.0, -1.6, 0.8, 0.0]
quantization forward : [0.0, -2.0, 0.0, 0.0]
```

(b) Execution result of the inheritance-based approach.

Figure 2: Inheritance-based implementation of the toy compression layer and its output.

C. Comparative Analysis

In Experiment 1, each compression case necessitates explicit conditional statements (`if/elif/else`). Conversely, Experiment 2 eliminates these branches by overriding the parent class methods. This allows the execution pipeline to invoke `layer.forward(2.0)` regardless of the specific layer type.

If a new technique such as KD-based compression is added, the procedural approach requires modifying the existing `forward` function, which increases the risk of regression errors. In contrast, the OOP approach only requires a new subclass. In addition, the pruning threshold is stored inside the `PrunedLayer` object, which makes the interface cleaner than repeatedly passing threshold values to one function.

4. Results and Discussion

The output of the basic layer is $[0.2, -1.6, 0.8, -0.4]$, which directly reflects the original weights multiplied by the input $x = 2.0$. In the pruning case, the output becomes $[0.0, -1.6, 0.8, 0.0]$ because the weights 0.1 and -0.2 do not satisfy the threshold condition and are therefore replaced with 0.0 before multiplication. In the quantization case, the output becomes $[0.0, -2.0, 0.0, 0.0]$ because the original weights are rounded to $[0, -1, 0, 0]$, showing that quantization reduces numerical precision and changes the resulting values.

These results demonstrate that compression methods intentionally modify the internal representation of the model and therefore affect the final output. However, the more important result of this experiment is structural rather than numerical. In the procedural implementation, every new compression method must be added to the same `forward` function through another conditional branch. This increases coupling and makes the code more difficult to maintain. In contrast, the inheritance-based implementation isolates each compression method in its own subclass while preserving the same `forward(x)` interface. Furthermore, the pruning threshold is stored inside the `PrunedLayer` object, which creates a cleaner and more robust interface than repeatedly passing configuration values to a single function. Therefore, although the OOP version is slightly longer in a small example, it is easier to extend, safer to modify, and more suitable for large-scale systems.

This design is also consistent with Python’s inheritance model and with the base-class structure used in PyTorch’s `nn.Module`.

5. Related Work

Han et al. (2016) proposed the “Deep Compression” pipeline, which sequentially applies pruning, trained quantization, and Huffman coding. Implementing such a complex, multi-stage pipeline using a procedural approach would result in an unmaintainable system due to the exponential increase in conditional logic. By utilizing the inheritance structure demonstrated in this report, researchers can encapsulate each technique into independent modules. This allows for the flexible assembly of compression pipelines without compromising the integrity of the original model.

6. Conclusion

This report showed that inheritance is more than a simple code reuse mechanism in Python OOP. By defining a common parent class and allowing subclasses to override the `forward()` method, the program can support multiple compression strategies through a uniform interface. Compared with the procedural approach, the inheritance-based design improves extensibility, reduces modification risk, and keeps implementation details localized in each subclass. For this reason, inheritance is a practical and effective design choice for building scalable software systems, especially when many related behaviors must coexist in the same framework.

References

Han, S., Mao, H., & Dally, W. J. (2016). *Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding*. In International Conference on Learning Representations (ICLR).

Python Software Foundation. (n.d.). *Classes*. Python documentation.

PyTorch Contributors. (n.d.). *torch.nn.Module*. PyTorch documentation.